Bi-Weekly Project Progress Report & Meeting Agenda

Meeting #3 - Phase 2: Finalization & Proposal Integration

Course:	03CIT4067 Cybersecurity Project	**Date:**	November 03, 2026 (Week 9)
Project Title:	Topic 15: Design and Implementation of a Web Application Security Risk Detection and Remediation Demonstration System Based on OWASP Top 10		
Team Leader:	WONG Lai Man (24810493)	**Supervisor:**	LIAM XU
Team Members:	Mustafa, Zoutao	**Meeting Phase:**	Phase 2 Final Integration & Evaluation

1. MEETING AGENDA

1. Review of Week 9 Internal Deliverables & DoD Validation (Chapter 5, Chapter 8, ZAP Report).
2. Project Proposal Integration (Assignment 03) mapping technical outcomes across 9 core sections.
3. Presentation Preparation & Live Demo Role Allocation (15-20 min limit, 5 min Q&A).
4. Blackboard Submission Compliance & Working Journal Individual Reflection (D4) rules.

2. SPRINT 3 REVIEW: WHAT WE HAVE DONE

2.1 A05: Security Misconfiguration Hardening

Baseline Vulnerability: Flask server was operating with `FLASK_DEBUG=1` enabled, exposing interactive Werkzeug debuggers and full stack traces. Missing key headers left users vulnerable to clickjacking and MIME-sniffing.

Execution & Remediation: Enforced `FLASK_DEBUG=0` across all environments. Added HTTP response headers via WSGI middleware (`Strict-Transport-Security`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Content-Security-Policy`) and suppressed server banners.

2.2 A10: Structured Audit Logging & Fail-Closed Infrastructure

Baseline Vulnerability: Authentication and access control events produced no persistent audit logs, creating severe monitoring blind spots during credential attacks.

Execution & Remediation: Implemented centralized JSON logging capturing ISO timestamps, client IP, route, HTTP method, and user ID. Applied regex sanitization filters to scrub passwords/tokens. Implemented fail-closed logic returning HTTP 403/500 without revealing backend stack traces.

3. UPDATED WBS & PROGRESS STATUS

WBS ID	Task Name	Assignee	Status
3.1	A05 Debug Mode & Security Headers Hardening	Mustafa	Completed
3.2	A10 Centralized JSON Audit Logging Implementation	Mustafa / WONG Lai Man	Completed
3.3	Red Team Chapter 5 Finalization & ZAP Scanning Report	Zoutao	Completed
3.4	Project Proposal (Assignment 03) APA 7th Integration	WONG Lai Man	Completed
3.5	Presentation Deck & Live Demo Rehearsal	All Present	Completed

4. BLOCKERS & TASK COORDINATION

Challenge & Resolution: Week 9 required smooth integration of red-team assessments and automated ZAP scans with backend remediation code. The team completed internal peer reviews by Nov 1 (23:59), eliminating version control conflicts and ensuring full compliance with Blackboard "One-Shot" submission rules.

5. ACTION ITEMS & NEXT STEPS

1. Finalize Project Proposal PDF (`AS_7_Proposal.pdf`) and Presentation Slides (`AS_7_Slides.pdf`).
2. Submit Individual Reflection (D4) and Working Journal bundle before Blackboard deadline.
3. Prepare and sign the Gen AI Declaration Form (`AS_7_GenAIForm.pdf`).

6. REFERENCES (APA 7TH FORMAT)

1. OWASP Foundation. (2025). *OWASP Top 10:2025: The ten most critical web application security risks*. https://owasp.org/www-project-top-ten/
2. OWASP Foundation. (2023). *OWASP Threat Modeling Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html

TECHNICAL APPENDIX: CUSTOM FLASK CODE IMPLEMENTATION (A05 HEADERS & A10 STRUCTURED LOGGING)

1. A05 Security Misconfiguration: Enforcing Security Headers & Debug Suppression

```
import os from flask import Flask, jsonify, request app = Flask(__name__) # Enforce strict non-debug mode across environments app.config['DEBUG'] = False @app.after_request def apply_security_headers(response): response.headers['X-Frame-Options'] = 'DENY' response.headers['X-Content-Type-Options'] = 'nosniff' response.headers['Content-Security-Policy'] = "default-src 'self';" response.headers['Strict-Transport-Security'] = 'max-age=31536000; includeSubDomains' response.headers.pop('Server', None) # Suppress server fingerprinting return response @app.errorhandler(500) def handle_internal_error(e): # Fail-closed generic error response suppressing stack trace return jsonify({"error": "Internal Server Error", "message": "An unexpected error occurred."}), 500
```

2. A10 Security Logging: Structured JSON Audit Trails & Fail-Closed Error Handling

```
import logging import json class JsonFormatter(logging.Formatter): def format(self, record): log_record = { "timestamp": self.formatTime(record), "level": record.levelname, "message": record.getMessage(), "ip": getattr(record, 'client_ip', 'N/A'), "path": getattr(record, 'request_path', 'N/A') } return json.dumps(log_record) logger = logging.getLogger("security_audit") handler = logging.StreamHandler() handler.setFormatter(JsonFormatter()) logger.addHandler(handler) logger.setLevel(logging.INFO) @app.route('/api/v1/auth/login', methods=['POST']) def login(): client_ip = request.remote_addr data = request.get_json() or {} username = data.get('username', 'unknown') # Audit logging for failed authentication if not data.get('password'): logger.warning(f"Failed login attempt for user: {username}", extra={'client_ip': client_ip, 'request_path': request.path}) return jsonify({"error": "Invalid credentials"}), 401 logger.info(f"Successful login for user: {username}", extra={'client_ip': client_ip, 'request_path': request.path}) return jsonify({"status": "Success"}), 200
```